

Rapport parallélisme

Nom: Bastien NGUYEN

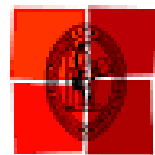
Numéro étudiant: 22300553

Groupe: B11

Exercice 4



UNIVERSITÉ
TOULOUSE III
PAUL SABATIER



Université
de Toulouse

Introduction

Pour réaliser cet exercice j'ai d'abord vérifié combien de cœur j'avais sur ma machine: 14 pour les machines de l'université et 6 pour la mienne. Tous les résultats que je montre seront faits à partir de ma machine.

Mon archive contient les fichiers de tests pb1, pb2, pb62 et les fichiers .c:

- ex4-seq pour la version séquentielle

- ex4-par pour la première version parallèle

Et un autre fichier ex4-moy qui prend toutes les versions et les compare en effectuant une moyenne de plusieurs de plusieurs exécutions avec le fichier pb6.txt. Je vais utiliser ce programme pour mes résultats.

J'ai paramétré ce dernier programme de manière à pouvoir choisir jusqu'à combien de thread on va tester les versions.

Par exemple si on met 6 alors mon programme va tester avec 1,2,3,4,5 et 6 thread pour les versions parallèles. De plus j'ai rajouté aussi un paramètre pour choisir le nombre d'exécution utilisé pour effectuer la moyenne. Si on choisit 3 alors 3 exécutions seront faites pour calculer une moyenne. Cela me permet d'avoir des résultats plus sûrs mais bien évidemment cela prend plus de temps car il faut calculer bien plus d'exécution.

Exemple d'exécution

```
splat@DESKTOP-MH7QE33:~/parallelisme/ex4$ ./ex4-moy 16 1
```

```
Temps d'exécution pour la version séquentielle  
-séquentielle: 0.360941
```

```
Temps d'exécution de la version parallèle avec 1 threads:  
-parallèle: 0.289862
```

```
Temps d'exécution de la version parallèle avec 2 threads:  
-parallèle: 0.156896
```

```
Temps d'exécution de la version parallèle avec 3 threads:  
-parallèle: 0.156186
```

```
Temps d'exécution de la version parallèle avec 4 threads:  
-parallèle: 0.132406
```

```
Temps d'exécution de la version parallèle avec 5 threads:  
-parallèle: 0.102954
```

```
Temps d'exécution de la version parallèle avec 6 threads:  
-parallèle: 0.097773
```

```
Temps d'exécution de la version parallèle avec 7 threads:  
-parallèle: 0.085848
```

```
Temps d'exécution de la version parallèle avec 8 threads:  
-parallèle: 0.075685
```

```
Temps d'exécution de la version parallèle avec 9 threads:  
-parallèle: 0.068857
```

```
Temps d'exécution de la version parallèle avec 10 threads:  
-parallèle: 0.064948
```

```
Temps d'exécution de la version parallèle avec 11 threads:  
-parallèle: 0.064992
```

```
Temps d'exécution de la version parallèle avec 12 threads:  
-parallèle: 0.081095
```

```
Temps d'exécution de la version parallèle avec 13 threads:  
-parallèle: 0.082522
```

```
Temps d'exécution de la version parallèle avec 14 threads:  
-parallèle: 0.082273
```

```
Temps d'exécution de la version parallèle avec 15 threads:  
-parallèle: 0.078459
```

Séquentielle

Pour faire la version séquentielle j'ai simplement suivi les instructions donné dans l'exercice. j'ai décidé de commencer les mesures de temps au moment ou on crée la matrice S et de le stopper quand on a finis de calculer le tableau E.

Comme pb6 est très grand on a besoin d'allouer dynamiquement S sinon cela créer des problèmes de mémoires.

Le temps d'exécution moyen avec pb6 est d'environ 0,309 secondes.

Dans mon fichier moy je vérifie que la version parralèle et séquentielle ont bien fait le bon calcul avec ma fonction Verify. Cette fonction marche uniquement pour pb6 c'est du bruteforce car je sais que tout les objets sont pris avec pb6. je ne la compte pas dans la mesure du temps elle est juste la pour vérifier que j'ai pas laissé une erreur quand je modifie mon programme.

Parallèle

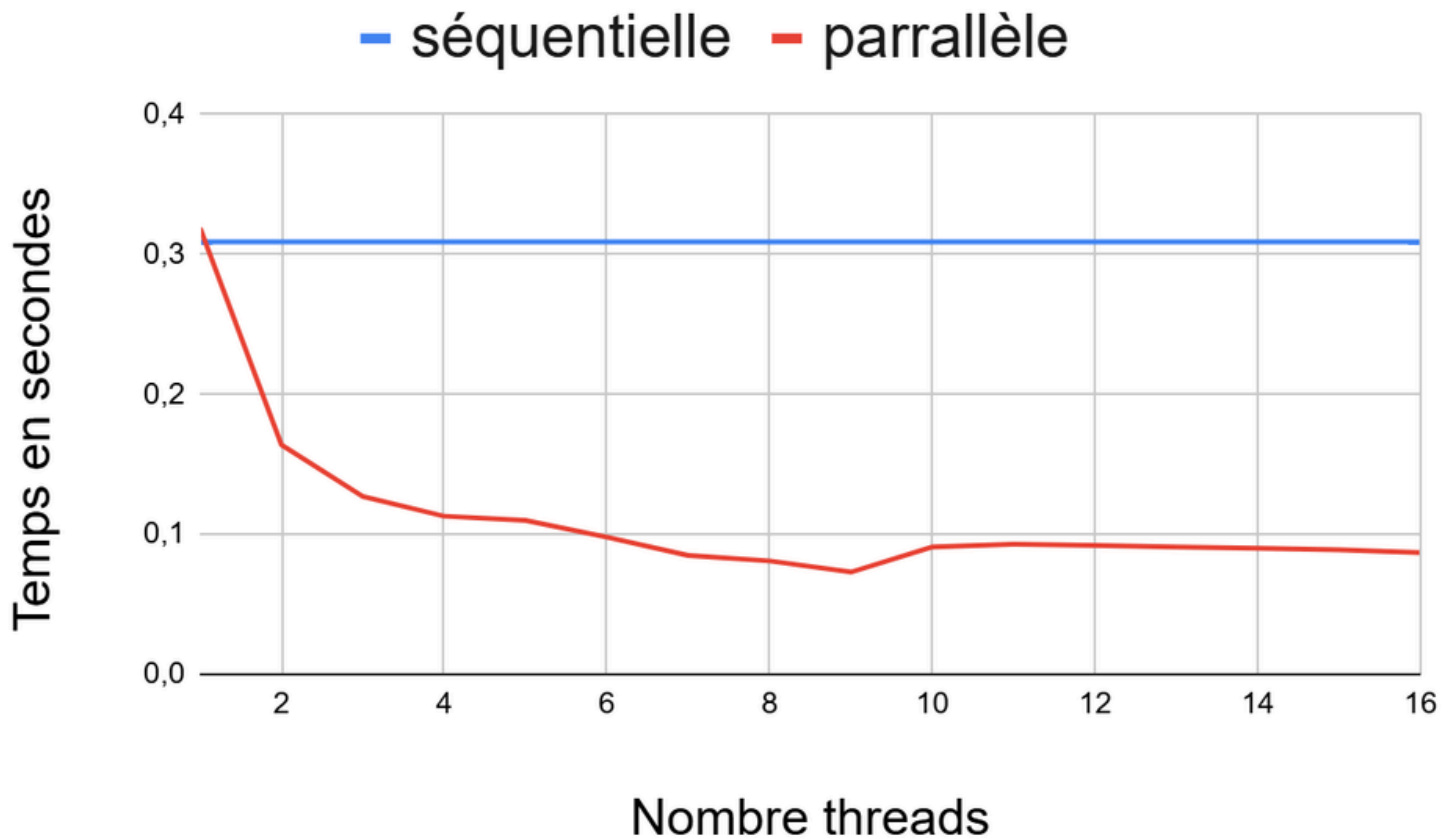
Pour réaliser la version parallèle, j'ai d'abord réfléchi à ce qui peut être paralléliser ou non. J'en ai conclu que le calcul de E n'est pas parallélisable car chaque itération de la boucle dépend de la précédente à cause de Cactu.

Pour la construction de S par contre plusieurs for sont parallélisable. Le premier qui s'occupe de la ligne 0 n'as aucune dépendance donc elle l'est. Le deuxième lui par contre a besoin que l'itération de la ligne précédente soit déjà faite mais pas les colonnes ce pourquoi on peut paralléliser le troisième for mais pas le deuxième.

Pour parralléliser ces for j'ai simplement utiliser un parallel for avec un nowait sur le premier for car pas besoin de la synchronisation mais sans nowait pour le troisième pour garder la synchronisation implicite car il y as une dépendance dans le calcul des lignes.

Comparaison

Voici une comparaison en temps entre séquentielle et parallèle.



On observe bien que jusqu'à mon nombre de cœur (6) l'accélération augmente (le temps d'exécution diminue) puis on commence à stagner comme attendu.

Pour calculer l'accélération on fait: $A(n) = T(\text{seq}) / T(\text{par } n)$.

$$A(2) = 1.88$$

$$A(4) = 2,73$$

$$A(8) = 3,81$$

$$A(16) = 3,55$$

Le calcul de l'accélération confirme bien que dépasser le nombre de cœur n'augmente plus l'efficacité du programme.